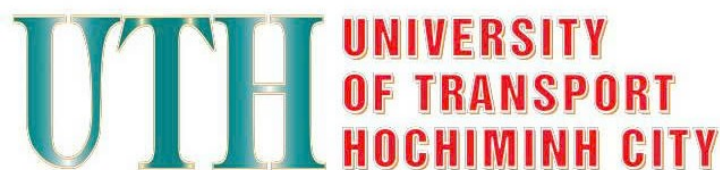


TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI  
TP. HỒ CHÍ MINH  
KHOA CÔNG NGHỆ THÔNG TIN



**BÁO CÁO HỌC PHẦN**

*Ngôn Ngữ Lập Trình Python*

**Nhóm 4**

**Đề Tài:**

Phân tích và so sánh GAN và các biến thể: DC-GAN,  
W-GAN và Q-GAN

**Giảng viên hướng dẫn: Ths. Nguyễn Văn Chiến**  
TP HỒ CHÍ MINH, 11 THÁNG 9 NĂM 2025

# Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành đến Thầy Nguyễn Văn Chiến, giảng viên học phần Ngôn ngữ lập trình Python, người đã tận tình giảng dạy và truyền đạt những kiến thức quý báu trong suốt quá trình học tập.

Những kiến thức mới mẻ và bổ ích từ Thầy đã giúp tụi em hiểu sâu hơn về xử lý dữ liệu cũng như cách vận dụng trong thực tiễn để giải quyết các mô hình bài toán. Tuy nhiên, do kiến thức và kinh nghiệm thực hành còn hạn chế, việc áp dụng của chúng em vẫn chưa thật sự tốt, và mong rằng được thầy hỗ trợ, tiếp tục nỗ lực để hoàn thiện hơn.

Xin trân trọng cảm ơn!

TP. Hồ Chí Minh, Ngày 11 Tháng 09 Năm 2025

**Thành viên và phân công nhóm 4**

| <b>STT</b> | <b>Họ và tên</b>     | <b>MSSV</b>  | <b>Vai trò</b>                          |
|------------|----------------------|--------------|-----------------------------------------|
| 1          | Trương Đỗ Vương      | 056206009261 | Viết báo cáo từ nội dung được cung cấp. |
| 2          | Nguyễn Hoàng Thái Kỳ | 054206001858 | W-GAN                                   |
| 3          | Nguyễn Văn An        | 075206007375 | GAN                                     |
| 4          | Hồ Đức Lực           | 052206012124 | Q-GAN                                   |
| 5          | Lê Nguyên Kiên       | 038206010272 | DC-GAN                                  |

# Mục lục

## DANH SÁCH HÌNH VẼ

## DANH SÁCH BẢNG

## DANH SÁCH BẢNG CHỮ CÁI VIẾT TẮT

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>GIỚI THIỆU</b>                                       | <b>1</b> |
| 1        | Bối cảnh . . . . .                                      | 1        |
| 2        | Mục tiêu . . . . .                                      | 1        |
| 3        | Các công trình liên quan . . . . .                      | 2        |
| 3.1      | Generative Adversarial Networks (GAN) . . . . .         | 2        |
| 3.2      | Deep Convolutional GAN (DC-GAN) . . . . .               | 2        |
| 3.3      | Wasserstein GAN (W-GAN) . . . . .                       | 2        |
| 3.4      | Q-GAN (Quantized GAN) . . . . .                         | 2        |
| 3.5      | Tổng kết . . . . .                                      | 2        |
| 4        | GAN cơ bản . . . . .                                    | 2        |
| 5        | DC-GAN . . . . .                                        | 3        |
| 6        | W-GAN . . . . .                                         | 3        |
| 7        | Q-GAN (Quantized GAN) . . . . .                         | 4        |
| 8        | Kết luận . . . . .                                      | 4        |
| <b>2</b> | <b>PHƯƠNG PHÁP NGHIÊN CỨU</b>                           | <b>5</b> |
| 1        | Các mô hình . . . . .                                   | 5        |
| 1.1      | Kiến trúc mô hình . . . . .                             | 5        |
| 1.1.1    | Kiến trúc của GAN . . . . .                             | 5        |
| 1.1.2    | Bảng tóm tắt số tham số và kích thước mô hình . . . . . | 6        |
| 1.1.3    | Cách tính số tham số & kích thước . . . . .             | 6        |
| 1.2      | Chiến lược huấn luyện và thu thập dữ liệu . . . . .     | 7        |
| 1.2.1    | Dữ liệu và tiền xử lý . . . . .                         | 7        |
| 1.2.2    | Hyperparameters chung . . . . .                         | 7        |
| 1.2.3    | Logging và lưu trữ lịch sử huấn luyện . . . . .         | 7        |
| 1.2.4    | Lưu ý cho W-GAN . . . . .                               | 7        |
| 2        | Dữ liệu và tiền xử lý . . . . .                         | 7        |
| <b>3</b> | <b>PHÂN TÍCH VÀ SO SÁNH</b>                             | <b>8</b> |
| 1        | Các metric so sánh . . . . .                            | 8        |
| 2        | Triển khai và thu thập dữ liệu . . . . .                | 8        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 2.1      | Môi trường . . . . .                                       | 8         |
| 2.2      | Chi tiết logging . . . . .                                 | 8         |
| 3        | Kết quả huấn luyện . . . . .                               | 8         |
| 3.1      | So sánh thời gian mỗi epoch giữa các mô hình . . . . .     | 9         |
| 3.2      | So sánh Loss_C / Loss_D (không bao gồm W-GAN) . . . . .    | 10        |
| 3.3      | So sánh Loss_G (không bao gồm W-GAN) . . . . .             | 11        |
| 3.4      | Phân tích W-GAN . . . . .                                  | 12        |
| 3.5      | So sánh số tham số và kích thước của các mô hình . . . . . | 12        |
| 3.6      | Biểu đồ chi tiết theo từng mô hình . . . . .               | 13        |
| 4        | Kết quả đầu ra . . . . .                                   | 14        |
| 5        | Tóm tắt và khuyến nghị . . . . .                           | 15        |
| <b>4</b> | <b>KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI</b>             | <b>16</b> |
| 1        | Ý nghĩa các quan sát . . . . .                             | 16        |
| 1.1      | Ý nghĩa giữa mô hình . . . . .                             | 16        |
| 1.2      | Hạn chế và hướng nghiên cứu tiếp theo . . . . .            | 16        |
| 2        | Kết luận . . . . .                                         | 17        |
|          | <b>DANH MỤC TÀI LIỆU THAM KHẢO</b>                         | <b>18</b> |

# Danh sách hình vẽ

|     |                                                                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | So sánh thời gian trung bình cho mỗi epoch giữa các mô hình. . . . .                                                                               | 9  |
| 3.2 | So sánh Loss_C / Loss_D theo epoch giữa GAN, DC-GAN và Q-GAN (W-GAN bị loại vì sử dụng raw score). . . . .                                         | 10 |
| 3.3 | So sánh Loss_G theo epoch giữa GAN, DC-GAN và Q-GAN. . . . .                                                                                       | 11 |
| 3.4 | Chi tiết kết quả huấn luyện của W-GAN: (bên trái) Loss_C (critic) theo epoch, (giữa) Loss_G theo epoch, (bên phải) thời gian/epoch (phút). . . . . | 12 |
| 3.5 | So sánh số tham số và kích thước lưu trữ của từng mô hình. . . . .                                                                                 | 13 |
| 3.6 | Bảng các kết quả huấn luyện theo từng mô hình (mỗi cái chứa 3 đồ thị nhỏ: Loss_D/C, Loss_G, Time/epoch). . . . .                                   | 13 |
| 3.7 | Kết quả tái tạo (reconstruction) cho mỗi mô hình. Mỗi ảnh là một lưới $4 \times 4$ (16 ảnh tái tạo). . . . .                                       | 14 |

# Danh sách bảng

|     |                                                                                          |    |
|-----|------------------------------------------------------------------------------------------|----|
| 2.1 | Kích thước các mô hình . . . . .                                                         | 6  |
| 2.2 | Lịch sử huấn luyện GAN trong file .csv . . . . .                                         | 7  |
| 3.1 | Số tham số và kích thước lưu trữ (theo bytes cho tham số, đã chuyển sang MB cho dễ đọc). | 12 |

# Danh sách bảng chữ cái viết tắt

| STT | TỪ VIẾT TẮT | TỪ TIẾNG ANH                                            | DỊCH NGHĨA                        |
|-----|-------------|---------------------------------------------------------|-----------------------------------|
| 1   | VAE         | Variational Autoencoder                                 | Bộ mã tự động biến thiên          |
| 2   | KL          | Kullback–Leibler Divergence                             | Phân kỳ Kullback–Leibler          |
| 3   | GAN         | Generative Adversarial Network                          | Mô hình đối kháng nghịch sinh     |
| 4   | DC-GAN      | Deep Convolutional Generative Adversarial Network       | GAN tích chập                     |
| 5   | W-GAN       | Wasserstein Generative Adversarial Network              | GAN Wasserstein                   |
| 6   | Q-GAN       | Quantized Generative Adversarial Network                | GAN lượng hoá                     |
| 7   | RE          | Reconstruction Error                                    | Lỗi tái cấu trúc                  |
| 8   | ADAM        | Adaptive Moment Estimation                              | Thuật toán tối ưu hoá Adam        |
| 9   | MNIST       | Modified National Institute of Standards and Technology | Tập dữ liệu chữ số viết tay MNIST |
| 10  | BCE         | Binary Cross-Entropy                                    | Hàm mất mát entropy chéo nhị phân |
| 11  | FID         | Fréchet Inception Distance                              | Khoảng cách Fréchet Inception     |
| 12  | LR          | Learning Rate                                           | Tốc độ học                        |
| 13  | GPU         | Graphics Processing Unit                                | Bộ xử lý đồ hoạ                   |
| 14  | CSV         | Comma-Separated Values                                  | Dữ liệu phân cách bằng dấu phẩy   |



# Chương 1

## GIỚI THIỆU

### 1 Bối cảnh

Trong vài năm gần đây, lĩnh vực mô hình tạo sinh (generative models) đã phát triển rất nhanh, mở ra nhiều ứng dụng thực tiễn như tạo ảnh, âm thanh, và nhiều ứng dụng khác; Trong số các kiến trúc thì Generative Adversarial Networks (GAN) đã tạo bước ngoặt nhờ khả năng tạo sinh với chất lượng cao.

Bên cạnh đó, nhiều biến thể đã được đề xuất nhằm khắc phục các hạn chế và nâng cao chất lượng tạo sinh. *DC-GAN* tận dụng kiến trúc convolutional để cải thiện chi tiết và cấu trúc không gian trong ảnh; *W-GAN* dùng khoảng cách Wasserstein và critic để đem lại gradients mượt hơn và độ ổn định trong huấn luyện tốt hơn; và các biến thể thực tiễn hơn như *Q-GAN* (*Quantized GAN*) hướng tới tiết kiệm bộ nhớ và tăng hiệu suất khi triển khai trên thiết bị biên thông qua lượng hóa trọng số/activation. Mỗi biến thể mang theo những điểm mạnh và yếu riêng giữa chất lượng đầu ra, độ đa dạng, tốc độ huấn luyện và chi phí tính toán.

Việc so sánh một cách có hệ thống giữa các mô hình trên cùng một bộ dữ liệu và trong cùng điều kiện thực nghiệm là cần thiết để hiểu rõ ưu-nhược điểm tương đối của từng phương pháp.

Một số khía cạnh then chốt được nhóm dùng để đánh giá các mô hình tạo sinh:

- **Chất lượng ảnh (output):** mức độ chân thực và chi tiết của ảnh sinh so với dữ liệu thật.
- **Đa dạng mẫu (diversity):** khả năng sinh ra nhiều dạng mẫu khác nhau.
- **Ổn định huấn luyện:** tính ổn định của quá trình tối ưu hóa (hội tụ, ít dao động, ít cần tuning hyperparameter).
- **Yêu cầu tài nguyên và hiệu năng khi triển khai:** kích thước mô hình, bộ nhớ, thời gian huấn luyện trên mỗi epoch; các kỹ thuật lượng hóa (Q-GAN).

Báo cáo này của nhóm tiến hành so sánh giữa GAN, DC-GAN, W-GAN và Q-GAN trên cùng điều kiện thí nghiệm để đưa ra nhận xét định lượng và định tính về từng khía cạnh nêu trên. Việc so sánh sẽ dựa trên các lịch sử huấn luyện (loss\_D, loss\_G, epoch\_time\_sec cho 300 epoch), phân tích về kích thước mô hình và thời gian huấn luyện nhằm cung cấp cái nhìn bổ sung khi lựa chọn mô hình trong các kịch bản ứng dụng khác nhau.

### 2 Mục tiêu

- So sánh chất lượng sinh ảnh/khả năng tái tạo giữa GAN, DC-GAN, W-GAN và Q-GAN.

- So sánh chi phí tính toán (thời gian huấn luyện theo epoch, tổng thời gian), và kích thước mô hình.
- Đưa ra khuyến nghị cho ứng dụng trên thiết bị biên/ nghiên cứu.

### 3 Các công trình liên quan

Sau đây, chúng em tóm tắt các công trình nền tảng liên quan trực tiếp tới đề tài: mô hình GAN cơ bản, biến thể DC-GAN, W-GAN và Q-GAN.

#### 3.1 Generative Adversarial Networks (GAN)

GAN, được giới thiệu bởi Goodfellow và các cộng sự, theo đó, sử dụng phương pháp đối kháng giữa hai mạng (generator và discriminator) cho thấy khả năng sinh mẫu chất lượng cao mà không cần mô hình hóa phân phối xác suất một cách tường minh như (VAE). Bài báo này là nền tảng cho hầu hết các nghiên cứu tiếp nối về GAN [Goodfellow, Pouget-Abadie, Mirza, Xu, Warde-Farley, Ozair, Courville and Bengio, 2014].

#### 3.2 Deep Convolutional GAN (DC-GAN)

Để tận dụng cấu trúc không gian của ảnh, Radford et al. đề xuất sử dụng các lớp convolution và transposed-convolution cho generator/discriminator — kiến trúc DC-GAN. DC-GAN cho ảnh sinh sắc nét, học được biểu diễn có ý nghĩa trong latent space và trở thành một trong những mô phổ biến cho các tác vụ ảnh [Radford, Metz and Chintala, 2016].

#### 3.3 Wasserstein GAN (W-GAN)

Arjovsky et al. trình bày W-GAN, đề xuất thay đổi hàm mất, sử dụng khoảng cách Wasserstein (Earth Mover's distance) thay vì Binary Cross-Entropy. Trong W-GAN, discriminator trở thành một *critic* trả về điểm thô (raw scores) thay vì xác suất; phương pháp này mong muốn giải quyết vấn đề như Vanishing Gradient và Mode [Arjovsky, Chintala and Bottou, 2017].

#### 3.4 Q-GAN (Quantized GAN)

Q-GAN nghiên cứu cách giảm bit-width cho trọng số/activation nhằm triển khai GAN lên thiết bị biên. Wang et al. trình bày một phương pháp lượng hóa chuyên biệt cho GAN dựa trên thuật toán EM và chiến lược multi-precision, cho kết quả khả quan trên CIFAR-10 và CelebA [P. Wang, D. Wang, Ji, X. Xie, Song, Liu, Lyu and Y. Xie, 2019].

#### 3.5 Tổng kết

Những công trình trên đặt nền tảng lý thuyết và thực nghiệm cho báo cáo này. Khi đánh giá và so sánh các mô hình (GAN / DC-GAN / W-GAN / Q-GAN), chúng em thống kê thời gian/kích thước mô hình để đảm bảo tính khách quan và khả năng so sánh.

## 4 GAN cơ bản

Generative Adversarial Networks (GAN), mô hình đối kháng nghịch sinh. Ý tưởng cốt lõi là đặt một *Generator*  $G$  và một *Discriminator*  $D$  trong một trò chơi tối ưu hóa hai người chơi (thuật toán Minimax

trong Game Theory):

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Trong thực nghiệm, người ta thường huấn luyện D trước để phân biệt ảnh thật và giả (với loss dạng binary cross-entropy — BCE), rồi huấn luyện G để lừa D (thông qua tối thiểu hóa  $\mathbb{E}[\log(1 - D(G(z)))]$  hoặc tối đa hóa  $\mathbb{E}[\log D(G(z))]$ ).

### Đặc điểm thực nghiệm

- Discriminator thường kết thúc bằng một activation **sigmoid** để trả xác suất; loss dùng BCE.
- Dễ xảy ra **mode collapse** (mô hình chỉ sinh một vài mẫu), và quá trình huấn luyện có thể bất ổn nếu D quá mạnh/ quá yếu.
- Thường dùng làm baseline; kiến trúc generator/discriminator có thể là fully-connected hoặc convolutional tùy ứng dụng.

## 5 DC-GAN

Deep Convolutional GAN (DC-GAN) là biến thể áp dụng các lớp convolution/deconvolution (transposed convolution) cho generator và discriminator.

### Ưu điểm kiến trúc

- Khả năng nắm bắt cấu trúc không gian của ảnh (local patterns) tốt hơn.
- Thường cho ảnh sinh sắc nét, các họa tiết cục bộ và cấu trúc hình học rõ ràng hơn.

### Lưu ý

- DC-GAN vẫn dùng loss BCE truyền thống (như GAN) — do đó các quan sát về loss là tương đối so sánh giữa các mô hình cùng kiểu loss.

## 6 W-GAN

Wasserstein GAN (W-GAN) ra đời nhằm khắc phục một số vấn đề về độ ổn định và gradients trong GAN. Thay vì tối ưu hóa divergence truyền thống, W-GAN sử dụng xấp xỉ khoảng cách Wasserstein (Earth Mover's distance). Đặc biệt: trong W-GAN, discriminator trở thành *critic*  $f$  trả về *raw scores* (không qua sigmoid). Mục tiêu tối giản cho critic và generator là:

$$\max_{f \in \mathcal{F}} \mathbb{E}_{x \sim p_{\text{data}}} [f(x)] - \mathbb{E}_{z \sim p_z} [f(G(z))], \quad \min_G -\mathbb{E}_{z \sim p_z} [f(G(z))]$$

với  $\mathcal{F}$  là tập các hàm 1-Lipschitz. Trong thực nghiệm cần thực hiện ràng buộc Lipschitz (ví dụ weight clipping trong W-GAN cơ bản hoặc *gradient penalty* trong WGAN-GP).

### Lưu ý

- **Raw critic scores không phải probabilistic outputs**  $\rightarrow$  không thể so sánh trực tiếp với BCE loss của GAN/DC-GAN.
- W-GAN thường cung cấp gradients mượt hơn, giúp huấn luyện ổn định hơn và giảm mode collapse.

## 7 Q-GAN (Quantized GAN)

**Quantized GAN** Là các mô hình GAN (Generator/Discriminator) đã được lượng hóa (quantization) về bit-width cho trọng số và/hoặc activation nhằm giảm kích thước mô hình và chi phí tính toán khi triển khai trên thiết bị biên.

### Ưu / nhược

- **Ưu:** giảm đáng kể dung lượng lưu trữ (MB), tăng throughput và giảm latency, phù hợp triển khai trên edge device.
- **Nhược:** Lượng hóa làm giảm chất lượng đầu ra của mô hình; Vì thế cần xem xét tỉ lệ đánh đổi giữa chất lượng và độ lớn của mô hình

### Các phân tích với Q-GAN

- Model size (MB) trước/sau lượng hóa; bit-width (e.g., float32  $\rightarrow$  int8).
- Thời gian huấn luyện mô hình.
- Generator loss, discriminator loss.

## 8 Kết luận

Các mô hình trình bày ở trên cung cấp trade-off rõ ràng giữa chất lượng ảnh, độ ổn định huấn luyện và chi phí tài nguyên: DC-GAN cải thiện chi tiết ảnh nhờ kiến trúc convolutional; W-GAN cải thiện độ ổn định và đưa ra tín hiệu loss có ý nghĩa hơn; Q-GAN là hướng thực dụng nhằm giảm dung lượng và tăng hiệu suất triển khai, nhưng cần cân trọng để giữ chất lượng sinh mẫu.

## Chương 2

# PHƯƠNG PHÁP NGHIÊN CỨU

## 1 Các mô hình

### 1.1 Kiến trúc mô hình

Trong báo cáo này, để so sánh một cách trực quan, chúng em thiết kế các mô hình có dung lượng tương đương về số tham số và cấu trúc đơn giản phù hợp với ảnh kích thước  $28 \times 28$  (MNIST). Dưới đây là mô tả kiến trúc cho mô hình GAN (bạn có thể xem các mô hình còn lại qua [Link GitHub](#), cách đếm số tham số và cách tính kích thước bộ nhớ.

#### 1.1.1 Kiến trúc của GAN

##### 1) Generator (vanilla / fully-connected)

```
1 class Generator(nn.Module):
2     def __init__(self, latent_dim):
3         super().__init__()
4         self.model = nn.Sequential(
5             nn.Linear(latent_dim, 128),
6             nn.ReLU(True),
7             nn.Linear(128, 256),
8             nn.ReLU(True),
9             nn.Linear(256, 28*28),
10            nn.Tanh()
11        )
12
13    def forward(self, z):
14        img = self.model(z)
15        img = img.view(z.size(0), 1, 28, 28)
16        return img
```

##### 2) Discriminator (vanilla / fully-connected)

```
1 class Discriminator(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.model = nn.Sequential(
5             nn.Flatten(),
6             nn.Linear(28*28, 256),
7             nn.LeakyReLU(0.2, inplace=True),
```

```

8         nn.Linear(256, 128),
9         nn.LeakyReLU(0.2, inplace=True),
10        nn.Linear(128, 1),
11        nn.Sigmoid() # output xác suất
12    )
13
14    def forward(self, img):
15        return self.model(img)

```

### 1.1.2 Bảng tóm tắt số tham số và kích thước mô hình

| Mô hình       | Số tham số | Kích thước (MB) |
|---------------|------------|-----------------|
| GAN (vanilla) | 481,425    | 1.84            |
| DC-GAN        | 499,350    | 1.90            |
| W-GAN         | 482,965    | 1.84            |
| Q-GAN         | 481,425    | 0.46            |

Bảng 2.1: Kích thước các mô hình

#### Giải thích các con số trong bảng

- **Số tham số:** được tính bằng tổng số weights và biases trong `state_dict` của mô hình (tức tổng kích thước tensor).
- **Kích thước (MB):** là kích thước lưu trữ của `state_dict` khi lưu dưới dạng float32 (mỗi tham số chiếm 4 bytes) cho các mô hình không lượng hóa. Giá trị 0.46 MB cho Q-GAN ở đây biểu thị kích thước sau khi *quantize* (int8), tức mỗi tham số chiếm 1 byte.

### 1.1.3 Cách tính số tham số & kích thước

```

1 def model_params_size_bytes(model):
2     params = sum(p.numel() * p.element_size() for p in model.parameters())
3     buffers = sum(b.numel() * b.element_size() for b in model.buffers())
4     return params + buffers
5
6 def n_parameters_approx(n_MB: float):
7     n_Bytes = int(n_MB * (1024**2))
8     n_parameters = n_Bytes // 4 #Cho rằng các tham số có kiểu dữ liệu float32
9
10    return int(n_parameters)
11
12 def bytes_to_mb(b):
13     return b / (1024**2)
14
15 generator_parameters_size = bytes_to_mb(model_params_size_bytes(generator))
16 discriminator_parameters_size = bytes_to_mb(model_params_size_bytes(discriminator))
17 total_size = generator_parameters_size + discriminator_parameters_size
18 print(f"Numbers of parameters: {n_parameters_approx(total_size):,}")
19
20 print(f"Model size (MB):\n\tGenerator: {generator_parameters_size:.2f} (MB)\n\tDiscriminator:
↪ {discriminator_parameters_size:.2f} (MB)\n\tTotal: {total_size:.2f} (MB)")

```

Tính kích thước tham số:

$$\text{size\_MB} \approx \frac{\text{num\_params} \times \text{bytes\_per\_param}}{10^6}$$

với `bytes_per_param = 4` cho `float32`, và `1` cho `int8`.

## 1.2 Chiến lược huấn luyện và thu thập dữ liệu

### 1.2.1 Dữ liệu và tiền xử lý

- Dataset: **MNIST** (28×28 grayscale). Tiền xử lý chuẩn: `ToTensor()` chuyển sang `[0,1]`, sau đó `Normalize(mean=0.5, std=0.5)` đưa giá trị về `[-1,1]` để nhất quán với đầu ra `Tanh()` của generator.
- Phân chia: dùng toàn bộ tập train cho huấn luyện.

### 1.2.2 Hyperparameters chung

- `latent_dim = 100`, `batch_size = 256` (khi dùng GPU), `epochs = 300`.
- Optimizer: Adam cho cả G và D, `lr = 3e-4`, `betas=(0.5, 0.999)`.

### 1.2.3 Logging và lưu trữ lịch sử huấn luyện

Mỗi epoch lưu:

- `loss_D` (trung bình trên batch trong epoch)
- `loss_G` (trung bình trên batch trong epoch)
- `epoch_time_sec` (cumulative) — thời gian chạy tích lũy trên mỗi epoch khi huấn luyện

Bảng 2.2: Lịch sử huấn luyện GAN trong file .csv

| Epoch | Loss_D | Loss_G | Epoch Time (s) |
|-------|--------|--------|----------------|
| 0     | 0.1816 | 2.3521 | 15.5212        |
| 1     | 0.0682 | 4.8892 | 31.4862        |
| ⋮     | ⋮      | ⋮      | ⋮              |
| 299   | 0.4522 | 1.7319 | 5545.1484      |

### 1.2.4 Lưu ý cho W-GAN

- W-GAN dùng *critic* trả *raw scores* (không qua sigmoid). Do đó giá trị `loss_D` không có cùng ý nghĩa số học như BCE loss của GAN/DC-GAN/Q-GAN.
- Trong thực nghiệm, áp dụng W-GAN + weight clipping.

## 2 Dữ liệu và tiền xử lý

- Dataset: MNIST (hoặc tập dữ liệu bạn dùng). Kích thước, phân chia train/val/test.
- Tiền xử lý: `ToTensor()`, `Normalize(0.5, 0.5)` → ảnh vào trong `[-1,1]`.

# Chương 3

## PHÂN TÍCH VÀ SO SÁNH

### 1 Các metric so sánh

- Generator Loss.
- Discriminator Loss.
- Thời gian huấn luyện.
- Dung lượng chiếm dụng của tham số.

### 2 Triển khai và thu thập dữ liệu

#### 2.1 Môi trường

- Môi trường phần mềm: Python3, PyTorch (tham khảo *requirements.txt*).

#### 2.2 Chi tiết logging

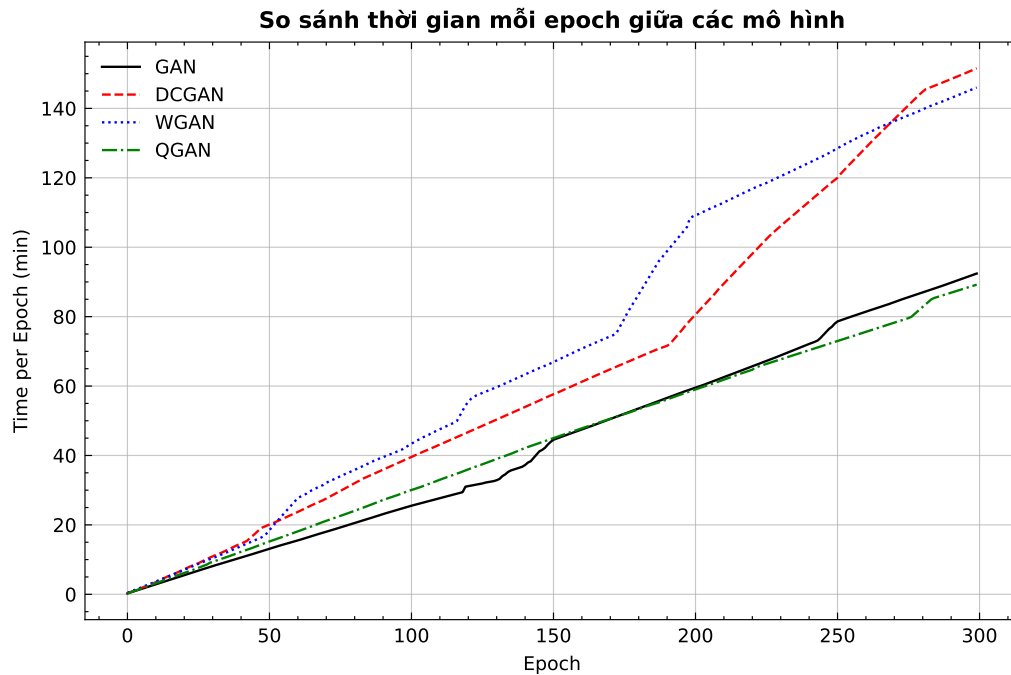
Cấu trúc file CSV đã lưu cho mỗi mô hình (ví dụ *models/training\_history/GAN\_training\_history.csv*):

### 3 Kết quả huấn luyện

Sau đây, nhóm xin trình bày kết quả huấn luyện và phân tích biểu đồ cho các mô hình GAN, gồm: GAN (vanilla), DC-GAN, W-GAN và Q-GAN.



### 3.1 So sánh thời gian mỗi epoch giữa các mô hình

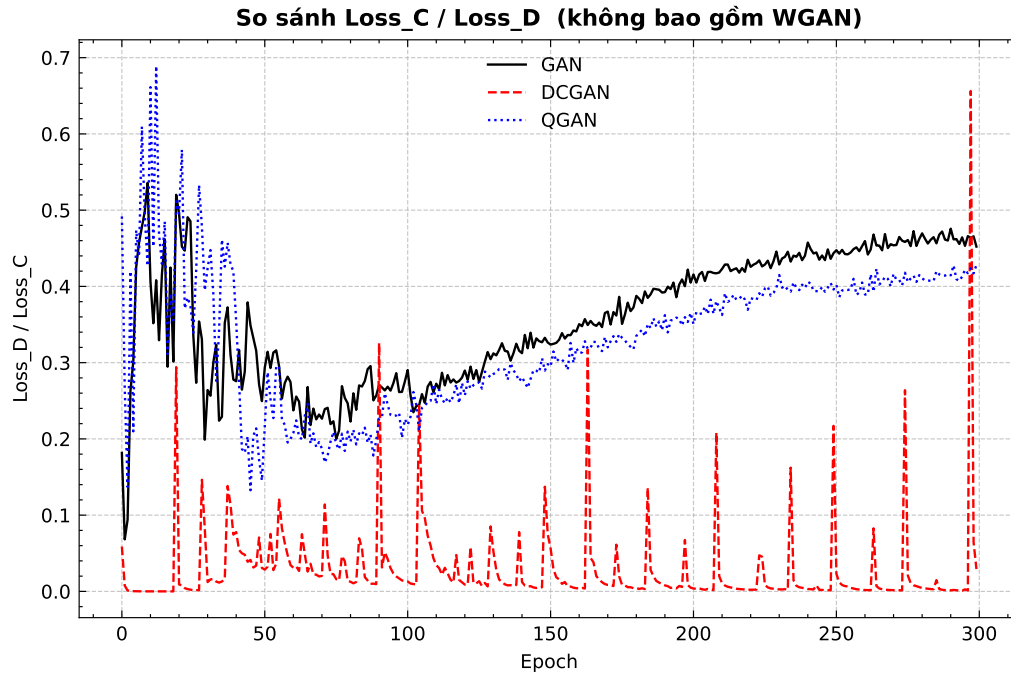


Hình 3.1: So sánh thời gian trung bình cho mỗi epoch giữa các mô hình.

Quan sát hình 3.1, thấy rằng:

- GAN và Q-GAN có xu hướng và thời gian huấn luyện khá tương đồng; tổng thời gian huấn luyện kết thúc vào khoảng  $\sim 95$  phút.
- W-GAN và DC-GAN tiêu tốn thời gian nhiều hơn; biểu đồ của chúng xuất hiện một số khoảng tăng rõ rệt trong thời gian huấn luyện. Tổng thời gian huấn luyện cho hai mô hình này kết thúc vào khoảng 145–150 phút.
- Những khoảng tăng đột ngột có thể do thay đổi trong việc epoch đó thực thi thêm bước tính toán (ví dụ cập nhật trọng số phức tạp hơn hoặc batch có kích thước biến đổi) hoặc giải phóng “bộ nhớ tạm”. Cần kiểm tra thêm, để xác định nguyên nhân chính xác.

### 3.2 So sánh Loss\_C / Loss\_D (không bao gồm W-GAN)

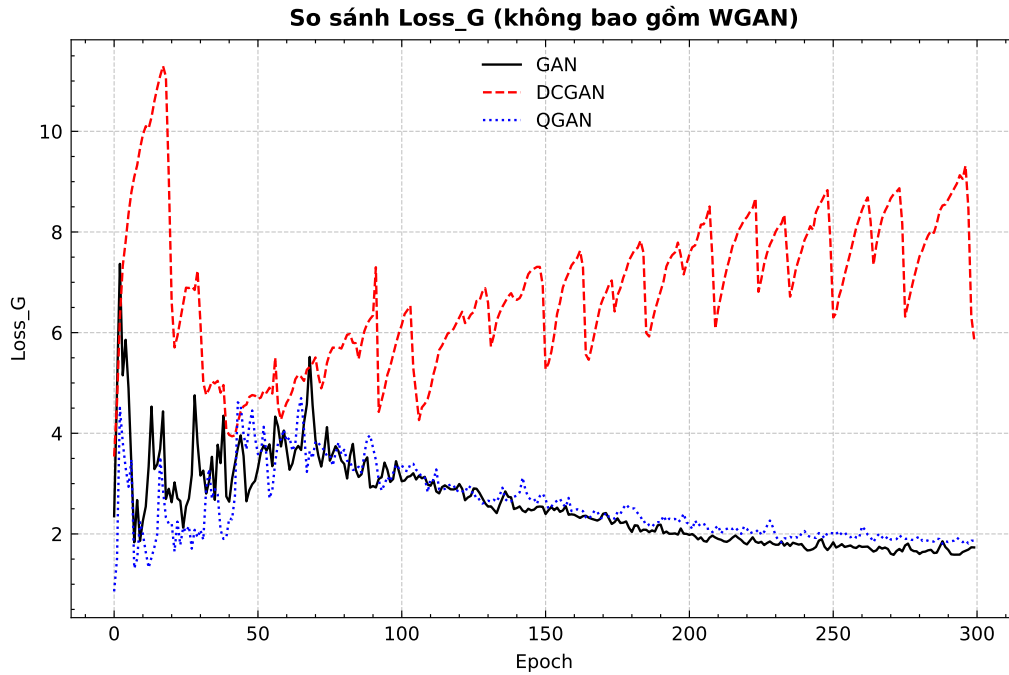


Hình 3.2: So sánh Loss\_C / Loss\_D theo epoch giữa GAN, DC-GAN và Q-GAN (W-GAN bị loại vì sử dụng raw score).

Quan sát chính (Hình 3.2):

- DC-GAN giảm loss rất nhanh xuống dưới 0.1 ở nhiều thời điểm, tuy nhiên biểu đồ cũng cho thấy các đợt nhô lên (spikes) khoảng 0.15–0.2 và có điểm cao nhất vào khoảng 0.65 (gần cuối quá trình huấn luyện). Điều này cho thấy DC-GAN có giai đoạn hội tụ nhanh nhưng đồng thời có dao động lớn (không hoàn toàn ổn định).
- GAN và Q-GAN gặp khó khăn hơn trong việc giảm loss xuống mức thấp: có nhiều dao động mạnh trong giai đoạn đầu (khoảng  $\sim 70$  epoch) rồi sau đó dần dần tăng trở lại — GAN kết thúc quanh 0.4 và Q-GAN là 0.45. Xu hướng này cho thấy cả hai mô hình có quá trình cân bằng giữa generator và discriminator/critic tương đối ổn định so với DC-GAN.

### 3.3 So sánh Loss\_G (không bao gồm W-GAN)

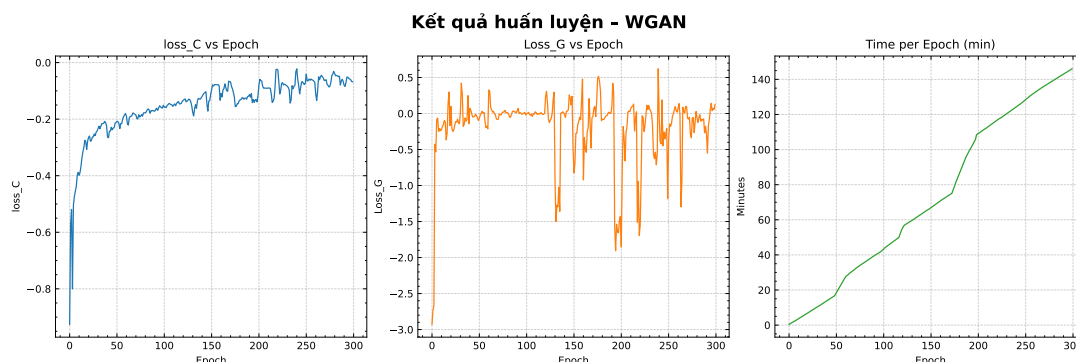


Hình 3.3: So sánh Loss\_G theo epoch giữa GAN, DC-GAN và Q-GAN.

Nhận xét (Hình 3.3):

- DC-GAN ban đầu rất bất ổn: tại những epoch đầu (khoảng  $< 20$  epoch) Loss\_G có giá trị rất cao (khoảng  $\sim 11$ ), sau đó giảm về khoảng 4 tại epoch  $\sim 50$ , và dao động tăng dần cho đến mức 6–8 vào cuối quá trình huấn luyện. Kết hợp với quan sát Loss\_C ở phần trước, DC-GAN tỏ ra không ổn định — đề xuất giảm learning rate hoặc áp dụng các kỹ thuật giúp ổn định mô hình và gradient (ví dụ weight clipping, gradient penalty, batch norm).
- Q-GAN và GAN ổn định hơn: cả hai bắt đầu ở mức Loss\_G khoảng 4 đến 6, GAN có đỉnh cao nhất khoảng 7.6 (ban đầu) trong khi Q-GAN có đỉnh thấp hơn (4.2). Cả hai dần cải thiện và đều đạt khoảng  $\sim 1.8$  ở cuối huấn luyện. Điều này cho thấy Q-GAN có phần ổn định hơn GAN ở một vài đoạn, nhưng tổng thể cả hai đạt được độ hội tụ tốt hơn DC-GAN trong điều kiện LR hiện tại.
- Kết luận tạm thời: DC-GAN cho thấy hội tụ nhanh nhưng kèm nhiều dao động lớn; điều chỉnh LR và/hoặc các kỹ thuật hỗ trợ khác có thể giảm sự bất ổn.

### 3.4 Phân tích W-GAN



Hình 3.4: Chi tiết kết quả huấn luyện của W-GAN: (bên trái) Loss\_C (critic) theo epoch, (giữa) Loss\_G theo epoch, (bên phải) thời gian/epoch (phút).

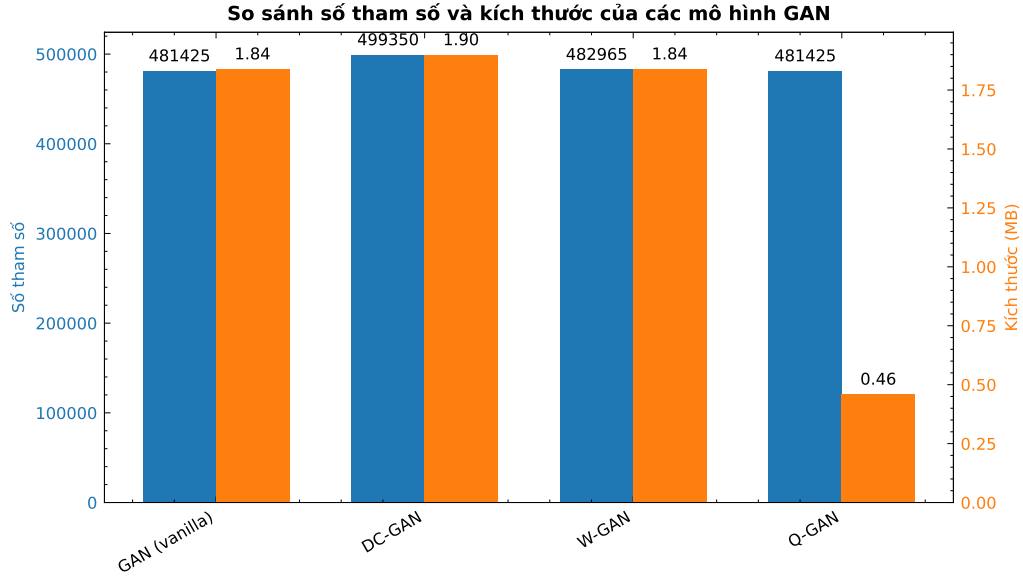
Phân tích (Hình 3.4):

- **Loss\_C (critic):** xu hướng tăng tương đối ổn định từ khoảng  $-1$  đến khoảng  $-0.06$ . Tốc độ tăng trong 50 epoch đầu nhanh hơn phần còn lại của quá trình huấn luyện, cho thấy giai đoạn đầu critic thay đổi mạnh hơn; sau đó critic tiếp tục tăng nhưng với đà chậm lại.
- **Loss\_G (generator):** rất bất ổn ở giai đoạn đầu — tăng từ  $-3$  lên khoảng  $0$  trong khoảng 10 epoch đầu, sau đó có giai đoạn tương đối ổn định kéo dài tới khoảng 120 epoch. Sau epoch 120, Loss\_G biểu hiện chập chờn thay đổi liên tục (ví dụ các giá trị như  $0.5, -2, -1$ ), rồi kết thúc gần  $0$  ở cuối quá trình huấn luyện. Cho thấy generator đôi khi bị mất ổn định nhưng dần phục hồi, có thể do tương giữa critic và generator hoặc do hyperparameter (LR, update ratio) không tối ưu.
- **Thời gian huấn luyện:** có một số dao động theo epoch và tổng thời gian huấn luyện kết thúc ở khoảng **146** phút — cùng với DC-GAN, W-GAN là một trong hai mô hình tốn thời gian nhất.
- **Nhận xét:** để giảm dao động của Loss\_G, có thể thử điều chỉnh tỉ lệ cập nhật (critic:generator update ratio), giảm learning rate cho generator, hoặc áp dụng/điều chỉnh gradient penalty / regularization cho critic.

### 3.5 So sánh số tham số và kích thước của các mô hình

Bảng 3.1: Số tham số và kích thước lưu trữ (theo bytes cho tham số, đã chuyển sang MB cho dễ đọc).

| Mô hình       | Số tham số | Kích thước (MB) |
|---------------|------------|-----------------|
| GAN (vanilla) | 481,425    | 1.84            |
| DC-GAN        | 499,350    | 1.90            |
| W-GAN         | 482,965    | 1.84            |
| Q-GAN         | 481,425    | 0.46            |



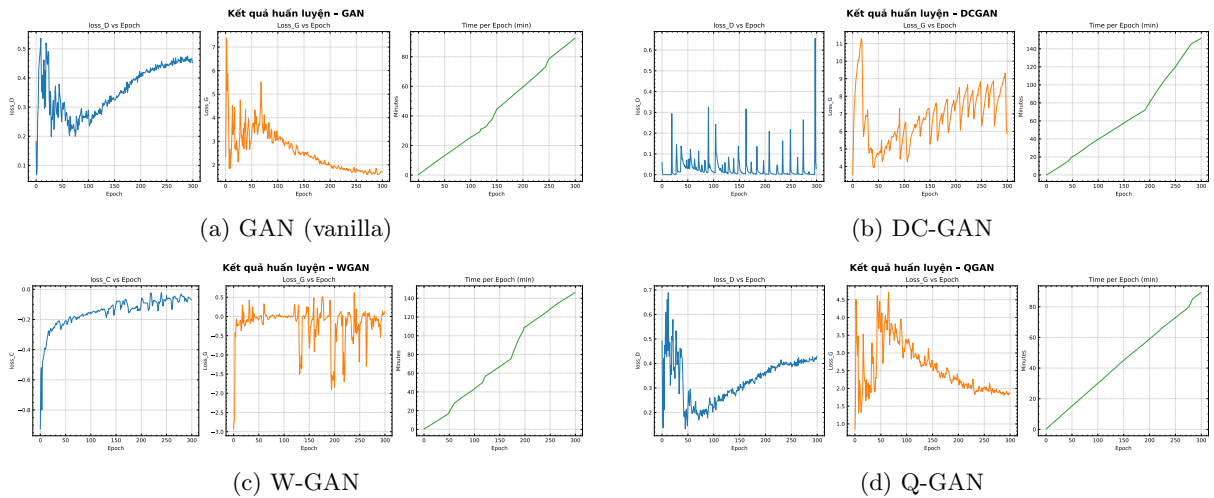
Hình 3.5: So sánh số tham số và kích thước lưu trữ của từng mô hình.

Nhận xét (Bảng 3.1, Hình 3.5):

- Tỷ lệ số tham số so với dung lượng lưu trữ của GAN, DC-GAN và W-GAN là tương tự nhau (khoảng  $\sim 1.8\text{--}1.9$  MB cho 482k tham số).
- Q-GAN nổi bật nhất: dù có số tham số tương đương với GAN, Q-GAN nhưng chỉ tiêu tốn  $\approx 0.46$  MB, cho thấy hiệu quả nén/quantization rất đáng kể. Điều này làm Q-GAN là lựa chọn tốt khi cần tiết kiệm bộ nhớ lưu trữ mà vẫn giữ kiến trúc tương tự.

### 3.6 Biểu đồ chi tiết theo từng mô hình

Mỗi mô hình được biểu diễn bằng một hình riêng, trong đó mỗi hình gồm ba đồ thị nhỏ: (i) Loss\_D / Loss\_C vs epoch, (ii) Loss\_G vs epoch, (iii) Time per epoch (min). Các đồ thị này tách rời từng mô hình giúp quan sát các đặc điểm chi tiết trong quá trình huấn luyện.

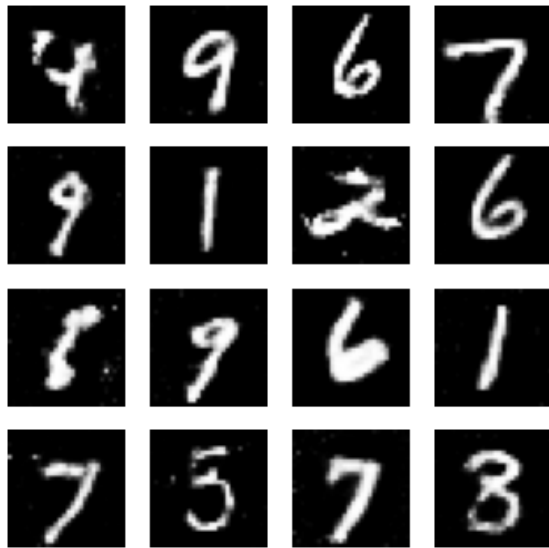


Hình 3.6: Bảng các kết quả huấn luyện theo từng mô hình (mỗi cái chứa 3 đồ thị nhỏ: Loss\_D/C, Loss\_G, Time/epoch).

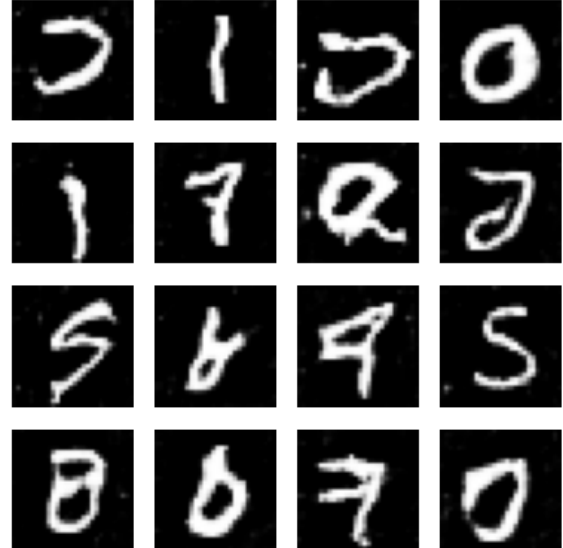
Nhận xét hình 3.6:

- Các nhận xét đã nêu ở phần trước được khẳng định ở mức mô hình riêng lẻ: DC-GAN thể hiện dao động lớn ở cả loss\_G và loss\_D; Q-GAN có độ ổn định tốt hơn và chi phí lưu trữ thấp nhất; W-GAN thường có thời gian huấn luyện cao và đôi khi xuất hiện các dao động về thời gian.
- Các dao động (trong cả loss và thời gian) cần được kiểm chứng thêm.

## 4 Kết quả đầu ra



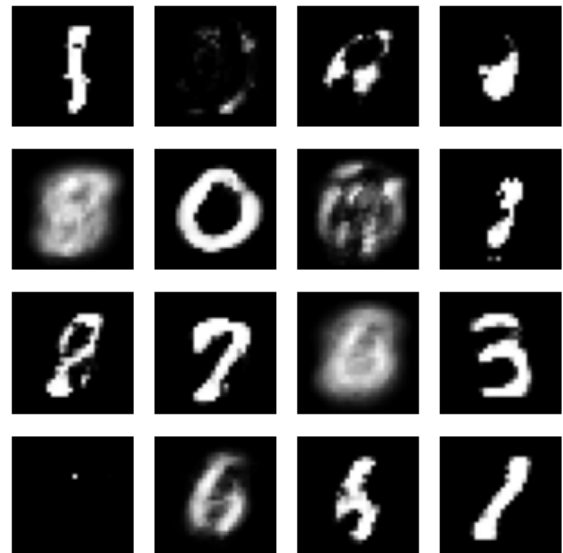
(a) GAN — 4×4 reconstruction grid (16 mẫu)



(b) DC-GAN — 4×4 reconstruction grid (16 mẫu)



(c) Q-GAN — 4×4 reconstruction grid (16 mẫu)  
mẫu



(d) W-GAN — 4×4 reconstruction grid (16 mẫu)

Hình 3.7: Kết quả tái tạo (reconstruction) cho mỗi mô hình. Mỗi ảnh là một lưới 4 × 4 (16 ảnh tái tạo).

Đánh giá chất lượng:

- **GAN (vanilla):** chất lượng tái tạo ở mức chấp nhận được; nhiều mẫu rõ ràng nhưng vẫn còn một số vùng bị mờ nhẹ. Tổng thể tương đồng với DC-GAN nhưng thường hơi kém hơn ở chi tiết vi mô.

- **DC-GAN:** thường cho kết quả tốt hơn GAN về độ sắc nét và chi tiết, tuy khác biệt không lớn — DC-GAN có phần ổn định hơn trên một số mẫu.
- **Q-GAN:** Ngạc nhiên rằng tái tạo của Q-GAN khá tốt (dù có vài mẫu bị mất nét hoặc thiếu chi tiết) nhưng còn lại thì tương đương với GAN/DC-GAN dù Q-GAN có giới hạn về kích thước bộ nhớ (quantized/compact). Điều này cho thấy phương pháp lượng tử hoá/ nén tham số đã giữ được chất lượng tái tạo ổn trong khi tiết kiệm đáng kể dung lượng lưu trữ.
- **W-GAN:** kém nhất trong nhóm. Độ không đồng nhất (inconsistency) lớn giữa các mẫu: ước lượng phân bố chất lượng là khoảng 40% các mẫu có chất lượng tương đương với các GAN khác, 50% mờ/failed reconstructions, và khoảng 10% là ảnh trống hoặc gần như không có thông tin. Kết hợp với các quan sát loss/thời gian, W-GAN cho thấy vấn đề ổn định trong huấn luyện và/hoặc vấn đề trong mô hình, cũng có thể là W-GAN không được thiết kế cho trường hợp trên. Dù sao thì việc sử dụng khoảng cách Wasserstein chủ yếu loại bỏ vấn đề Discriminator áp đảo Generator từ đó dẫn đến Vanishing Gradient; Do vậy, với bộ dữ liệu đơn giản như MNIST, khả năng gặp vấn đề này là không cao, điều đó phần nào khiến cho việc sử dụng W-GAN trên MNIST là dư thừa [Arjovsky, Chintala and Bottou, 2017].

## 5 Tóm tắt và khuyến nghị

- **Hiệu năng thời gian:** GAN và Q-GAN là nhanh nhất (kết thúc  $\sim 90$  phút); W-GAN và DC-GAN tiêu tốn thời gian hơn (kết thúc  $\sim 145$ – $150$  phút) và có các spike không mong muốn.
- **Ổn định huấn luyện:** DC-GAN hội tụ nhanh ở một số giai đoạn nhưng kèm dao động lớn (cần tuning LR/regularization). GAN và Q-GAN ổn định hơn về loss\_G; Q-GAN có xu hướng ổn định hơn GAN trong một vài đoạn.
- **Kích thước mô hình:** Về tham số, hầu hết mô hình có số tham số tương đương nhau; Q-GAN nổi bật ở chỗ sử dụng bộ nhớ nhỏ hơn đáng kể (0.46 MB) — phù hợp cho triển khai có hạn chế bộ nhớ.
- **Khuyến nghị:** Thực hiện profiling chi tiết trên epoch có spike; thử giảm learning rate cho DC-GAN, thêm gradient penalty / weight regularization nếu cần; khi cần triển khai nhẹ, ưu tiên Q-GAN.

## Chương 4

# KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI

### 1 Ý nghĩa các quan sát

Điểm cuối là tổng hợp và phân tích các quan sát thu được từ quá trình huấn luyện. Mục tiêu là rút ra những kết luận có ý nghĩa thực tiễn cho việc thiết kế, huấn luyện và triển khai các biến thể GAN được đề cập.

#### 1.1 Ý nghĩa giữa mô hình

- **GAN (vanilla) vs DC-GAN.** Cả hai có số lượng tham số gần nhau; nếu DC-GAN cho loss ổn định hơn hoặc hội tụ nhanh hơn (bằng cách điều chỉnh các siêu tham số như batch size, LR, ...) thì điều đó chỉ ra rằng thay đổi kiến trúc (ví dụ conv, conv transpose, batchnorm, ...) thực sự giúp điều chỉnh gradient và cải thiện huấn luyện. Còn nếu thời gian/epoch chỉ tăng nhẹ ở DC-GAN, đó là chi phí chấp nhận để cải thiện chất lượng.
- **W-GAN (critic).** Vì W-GAN dùng hàm mất mát khác (Wasserstein) và thường đòi hỏi cập nhật critic nhiều lần hoặc gradient penalty, so sánh loss giữa W-GAN và các GAN khác là không phù hợp. Nếu đường loss của W-GAN mượt hơn và loss G/ loss C giảm ổn định, đó là bằng chứng về lợi ích của hàm mục tiêu Wasserstein trong việc tăng cường tính ổn định huấn luyện.
- **Q-GAN (kích thước nhỏ).** Dữ liệu cho thấy Q-GAN có kích thước lưu trữ (MB) nhỏ hơn nhiều trong khi số tham số tương đương với GAN vanilla — điều này cho thấy khả năng lượng hoá (quantization) hiệu quả. Ý nghĩa thực tiễn: Q-GAN thích hợp cho triển khai trên thiết bị biên (edge) hoặc môi trường có hạn chế bộ nhớ. Tuy nhiên cần kiểm tra chất lượng mẫu kỹ hơn: nếu chất lượng đầu ra chấp nhận được hoặc tốt hơn, Q-GAN là lựa chọn hiệu quả về chi phí; nếu chất lượng giảm nhiều, cần cân nhắc trade-off.

#### 1.2 Hạn chế và hướng nghiên cứu tiếp theo

- **Hạn chế:** Những kết luận trên chủ yếu dựa trên đồ thị loss, thời gian/epoch và kích thước mô hình. Để khẳng định hiệu năng sinh ảnh cần bổ sung các tiêu chí định lượng (FID, IS) và đánh giá định tính.
- **Hướng tiếp theo:**



- Đánh giá chất lượng sinh ảnh bằng FID/IS
- Thử nghiệm nhiều lần để có ước lượng chính xác hơn
- Kiểm tra các cấu hình Q-GAN khác nhau (bitwidth) để tìm trade-off tối ưu giữa kích thước và chất lượng
- Kiểm tra và tối ưu hóa inference latency để triển khai thực tế.

## 2 Kết luận

Những quan sát thu thập được cho thấy mỗi biến thể GAN mang một ưu và nhược điểm giữa ổn định huấn luyện, chất lượng sinh ảnh, chi phí tính toán và kích thước mô hình. Việc lựa chọn mô hình “tốt nhất” phụ thuộc vào mục tiêu cuối cùng cả người triển khai.

# Tài liệu tham khảo

- [1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville **and** Y. Bengio, *Generative Adversarial Networks*, 2014. arXiv: [1406.2661 \[stat.ML\]](#). url: <https://arxiv.org/abs/1406.2661>.
- [2] A. Radford, L. Metz **and** S. Chintala, *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*, 2016. arXiv: [1511.06434 \[cs.LG\]](#). url: <https://arxiv.org/abs/1511.06434>.
- [3] M. Arjovsky, S. Chintala **and** L. Bottou, *Wasserstein GAN*, 2017. arXiv: [1701.07875 \[stat.ML\]](#). url: <https://arxiv.org/abs/1701.07875>.
- [4] P. Wang, D. Wang, Y. Ji, X. Xie, H. Song, X. Liu, Y. Lyu **and** Y. Xie, *QGAN: Quantized Generative Adversarial Networks*, 2019. arXiv: [1901.08263 \[cs.NE\]](#). url: <https://arxiv.org/abs/1901.08263>.